COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	July 22, 2026	**DATE SUBMITTED:**	July 22, 2026
NAME:	Jaime Luis M. Demain	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

SEATWORK NO. 3.1 Creating a Singly Linked List

1. PROCEDURE OUTPUT

What procedures did you perform?

```
#include <iostream>
#include "singly_ll.h"

int main() {

    //creation of nodes "JLMD"
    SingleList<char>* head = nullptr;
    SingleList<char>* second = nullptr;
    SingleList<char>* third = nullptr;
    SingleList<char>* fourth = nullptr;

    //allocate 4 nodes in the heap
    head = new SingleList<char>();
    second = new SingleList<char>();
    third = new SingleList<char>();
    fourth = new SingleList<char>();

    //store data in nodes
    head->data = 'J';
    second->data = 'L';
    third->data = 'M';
    fourth->data = 'D';

    //connect nodes
    head->next = second; //connect first node with the second node
    second->next = third; //connect second node with the third node
```

```
    third->next = fourth; //connect third node with the fourth node
    fourth->next = nullptr; //connect fourth node with the null pointer

    std::cout << "Testing of Traversal:\n";
    ListTraversal(head);

    // Test insertion at head
    std::cout << "\nTesting of sllinsertHead:\n";
    sllinsertHead('D', &head);
    ListTraversal(head);

    std::cout << "\nTesting of sllGeneralInsert:\n";
    sllGeneralInsert('X', head->next->next); // Insert 'X' after the
second node
    ListTraversal(head);

    std::cout << "\nTesting of sllinsertEnd:\n";
    sllinsertEnd('Z', &head); // Insert 'Z' at the end of the list
    ListTraversal(head);
    std::cout << "\nTesting of deleting the node:\n";
    sllDeleteNode('M', &head);
    ListTraversal(head);

    std::cout << "\nDeleting All nodes:\n";
    sllDeleteList(&head);
    ListTraversal(head);

    return 0;
}
```

Code for .cpp file

```cpp
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

template <typename T>
class SingleList {
    public:
    T data; //this contains the data of the node
    SingleList<T>* next = nullptr; //this contains the pointer to the
next node
};

template <typename T>
void ListTraversal(SingleList<T>* head) {
    while (head != nullptr) {
        //PRINT data of n
        std::cout << head->data;
        if (head->next != nullptr) {
        std::cout << " -> ";
        }
        //GO TO NEXT NODE n := next
        head = head->next;
    }
}

template <typename T>
void sllinsertHead(T newData, SingleList<T>** currentHead) {
    //1. Allocate memory for new node
    SingleList<T>* newNode = new SingleList<T>();
    //2. Put our data into the new node
    newNode->data = newData;
    //3. Set Next of new node to point to the previous head
    newNode->next = *currentHead;
    //4. Reset head to point to the new node
    *currentHead = newNode;
}

template <typename T>
void sllGeneralInsert(T newData, SingleList<T>* preNode) {
```

```
    //1. Check if the given previous node is NULL
    if (preNode == nullptr) {
        //2. If the given previous node is NULL, we cannot insert a new
node after it
        std::cout << "The given previous node cannot be NULL";
        return;
    }
    //3. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>();
    //4. Store data in the new node
    newNode->data = newData;
    //5. Point new node's next to the previous node's next
    newNode->next = preNode->next;
    //6. Point previous node to the new node
    preNode->next = newNode;
}

template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head){
    //1. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>();
    //2. Dereference to the head node
    SingleList<T>* currentNode = *head;
    //3. Store data in the new node
    newNode->data = newData;
    //4. Point to Null
    newNode->next = nullptr;
    //5. Traverse the list to find the last node
    while (currentNode ->next != nullptr) {
        currentNode = currentNode->next;
    }
    //6. Point the last node's next to the new node
    currentNode->next = newNode;
}

template <typename T>
void sllDeleteNode(T findData, SingleList<T>** head) {
```

```
    //if there is nothing to be deleted?
    if (*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    //search for the node to be deleted
    while (currNode != nullptr && currNode->data != findData) {
        prevNode = currNode;
        currNode = currNode->next;
    }
    //if the node to be deleted is not found
    if (currNode == nullptr) {
        std::cout << "the data is not found  \n";
        return;
    }

    //if data was found, delete the node
    if (prevNode == nullptr) {
        //the node to be deleted is the head node
        *head = currNode->next;
    } else {
        //the node to be deleted is not the head node
        prevNode->next = currNode->next;
    }

    delete currNode; //free memory
}

template <typename T>
void sllDeleteList(SingleList<T> **head){
    SingleList<T>* current = *head;

    while (current != nullptr) {
        SingleList<T>* temp = current;
        delete current; // Free memory of the current node
        current = current->next; // Move to the next node
        delete temp; // Free memory of the temporary node
```

```
    }

    *head = nullptr; // Set head to nullptr after deletion
}

#endif
```

Code for header file

```
PS C:\Users\jaime\OneDrive\Documents\HOA> cd "c:\Users\jaime\OneDrive\Documents\HOA\hoa3p1\" ; if ($?) { g++ hoa3.cpp -o hoa3 } ; if ($?) { .\hoa3 }
Testing of Traversal:
J -> L -> M -> D
Testing of sllinsertHead:
D -> J -> L -> M -> D
Testing of sllGeneralInsert:
D -> J -> L -> X -> M -> D
Testing of sllinsertEnd:
D -> J -> L -> X -> M -> D -> Z
Testing of deleting the node:
D -> J -> L -> X -> D -> Z
Deleting All nodes:
PS C:\Users\jaime\OneDrive\Documents\HOA\hoa3p1>
```

Output of the program

Conclusion: This activity allowed me to gain insight into how the dynamic link list works in terms of memory usage. Through the use of raw pointers and node objects, I gained knowledge on how to allocate memory dynamically for each node, how to create memory addresses chains that allow creating a sequence and how pointers can be redefined to achieve insertions from the head of the list, insertion at any node and deletions. Apart from simple manipulations, the analysis of the traversal and deallocation algorithms revealed how important is proper use of pointers in C++ especially in terms of edge cases and de-allocation of the memory.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar improvement on forming the conclusion**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **I used AI to fix any grammar mistakes in the conclusion part. I confirm that I did not use AI to simply copy and paste the conclusion; it just helped me to get my own work a refined version, which mostly improved its grammar and tone.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.